InstAgent: A Recursively Self-Improving Agent Operating System for Heterogeneous Robot Collaboration

Haoyang Luo[1,2,*], Junhao Qin[1,2,*], Jingyan Cao[1,2*], Yi Huang[1], Guangjie Yang[1], Xiaoyu Chen[1], Haotao Lu[1,2], Tianxiang Gui[1,2], Ye Shi[1,2], and Jingya Wang[1,2,†]

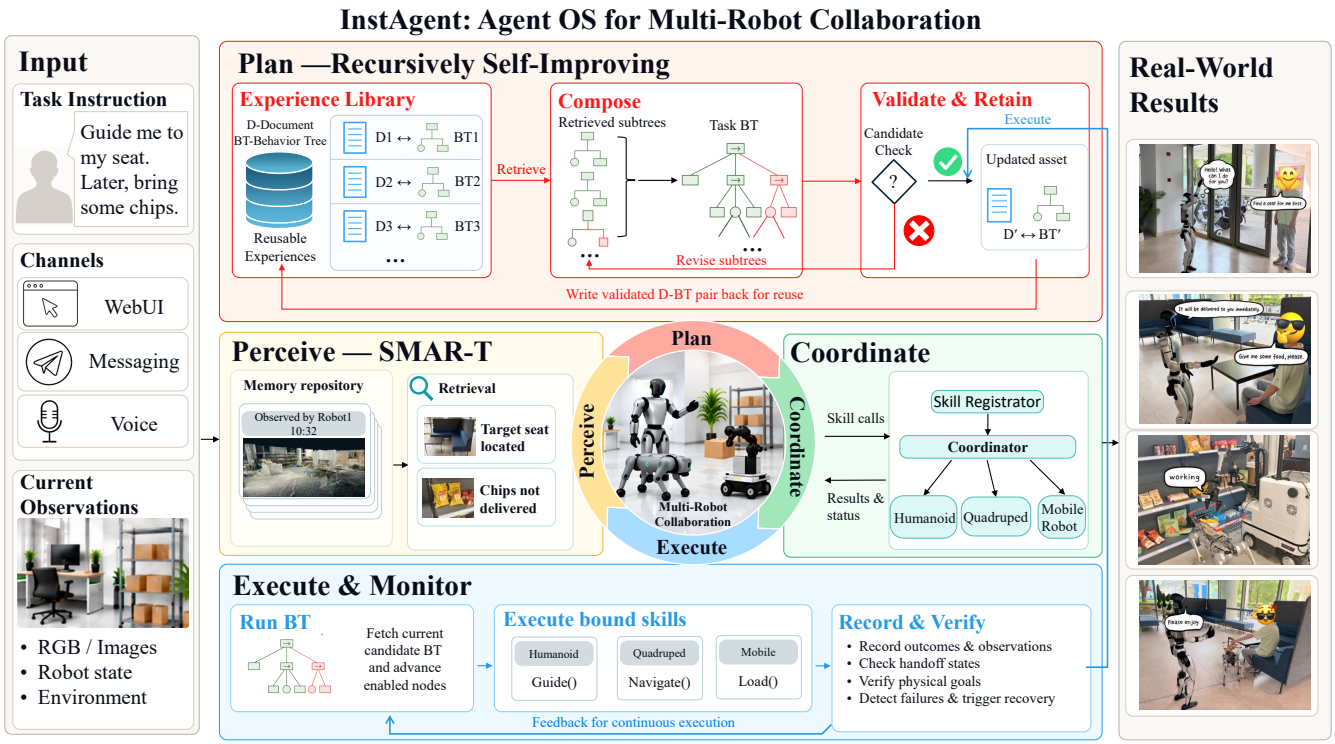


Fig. 1. Overview of **InstAgent**, a recursively self-improving Agent OS for heterogeneous multi-robot collaboration. InstAgent forms a closed **Perceive–Plan–Coordinate–Execute** loop: shared observations provide task context, reusable experience is composed into collaborative behavior trees, robot capabilities are coordinated through explicit dependencies and handoffs, and execution outcomes are continuously verified. Execution feedback revises failed collaboration structures, while validated revisions are retained as reusable document–BT pairs for future inheritance and adaptation.

***Abstract*— Heterogeneous robot teams enable complex, long-horizon tasks by combining complementary sensing, mobility, and manipulation capabilities across multiple embodiments. Yet existing systems struggle to retain execution-time corrections as reusable experience, maintain task-relevant context memory across space and time for reliable handoffs, and explicitly represent the inter-robot dependencies required for structured collaboration. In order to address these challenges, we present InstAgent, a recursively self-improving (RSI) Agent Operating System for heterogeneous robot collaboration. InstAgent organizes team behaviors as agentic behavior trees(BTs) that explicitly model capability assignment, inter-robot dependencies and handoff checks. The RSI mechanism updates these collaboration programs from execution feedback and preserves successful revisions for reuse in future tasks, while shared multimodal spatio-temporal memory provides context for planning and handoff validation. Experiments on 20 real-world multi-robot tasks with three high-level models show consistent gains in task success, first-attempt completion, and execution efficiency.**

*Haoyang Luo, Junhao Qin, and Jingyan Cao contributed equally to this work.
[1]ShanghaiTech University, Shanghai, China.
[2]InstAdapt, Shanghai, China.
†Corresponding author: Jingya Wang.

I. Introduction

Heterogeneous robot teams combine complementary capabilities in perception, mobility and manipulation, enabling long-horizon tasks which are difficult for a single embodiment to complete. Such tasks, however, require much more than assigning individual skills to different robots. Robots operate across different locations and stages, and the outcome of one robot may determine whether another can proceed. Recent large-language-model-based agentic systems provide a promising abstraction for this setting by connecting high-level reasoning, heterogeneous skill invocation, multimodal

perception, and execution feedback in a closed loop. This makes them particularly suitable for long-horizon heterogeneous collaboration, where task procedures cannot always be fully specified in advance.

Despite this promise, three challenges remain. First, complex physical tasks rarely succeed from an initial plan alone, yet execution corrections are often task-specific rather than accumulated as reusable executable experience. Existing work has explored reflection, skill reuse, and feedback-driven adaptation [1]–[4], but collaboration procedures corrected during execution are not always retained in a form that later tasks can inherit and further improve. Second, long-horizon collaboration requires persistent spatio-temporal context beyond current local observations, which helps heterogeneous robots plan and act accordingly. Existing memory systems support spatial and temporal retrieval [5]–[8], but historical observations must still be connected to current physical checks before inter-robot handoffs. Third, treating heterogeneous robots as a flat collection of callable skills underspecifies collaboration structure. Reliable execution requires explicit inter-robot dependencies, handoff conditions, and recovery logic, rather than isolated skill invocation alone [9], [10].

To address the aforementioned challenges, we present **InstAgent**, a recursively self-improving Agent Operating System designed to enable instant adaptation and deployment of heterogeneous robot teams across diverse and complex scenarios. As shown in Fig. 1, InstAgent represents collaboration as an executable behavior-tree (BT) program that can be grounded in shared environmental context, verified during physical execution, and improved through accumulated experience. Its BT-based recursive self-improvement (RSI) revises failed collaboration structures from execution feedback and retains validated revisions for inheritance and further adaptation. Shared Multimodal Spatio-Temporal Memory with Agent-Driven Retrieval (SMAR-T) provides task-relevant observations with spatial, temporal, and source information, while current observations verify physical handoff conditions. InstAgent further composes heterogeneous capabilities into BTs that explicitly encode inter-robot dependencies, handoff conditions, and recovery branches. Together, these components form a closed loop of composition, execution, verification, revision, and reuse.

We evaluate InstAgent on real-world embodiments across 20 heterogeneous multi-robot tasks and three high-level models. Compared with BT execution without cross-task procedural experience, feedback-updated experience improves task success by 15–25 percentage points across the three models, while increasing first-attempt success and reducing recoveries. In the meanwhile, RAVEN-QA independently evaluates task-relevant environmental observation retrieval.

Our main contributions are:

- An agentic OS for heterogeneous multi-robot collaboration that organizes robot capabilities and inter-robot dependencies into executable BTs for structured coordination and execution.
- An RSI mechanism that revises collaboration programs from execution feedback and retains validated revisions for reuse, supported by shared multimodal spatio-temporal memory (SMAR-T).
- Extensive real-robot evaluations across heterogeneous platforms, tasks, and high-level models, demonstrating consistent improvements in task success and execution performance.

TABLE I
CAPABILITY COMPARISON OF AGENTIC ROBOTIC SYSTEMS.
✓: EXPLICITLY REPORTED; ×: NOT EXPLICITLY REPORTED.

System	Collab.	Spatial memory	RSI	
			Experience-Context	Structured-Program
LRLL [3]	×	×	✓	✓
LLM-HBT [10]	✓	×	×	×
PhyAgentOS [4]	×	×	✓	×
ABot-AgentOS [15]	×	✓	✓	×
HoloAgent-0 [7]	✓	✓	✓	×
ABot-Claw [8]	✓	✓	✓	×
InstAgent	✓	✓	✓	✓

II. RELATED WORK

A. Physical Agents and Embodied Execution

Physical-agent systems connect language reasoning with physical execution through programs, robot interfaces, and environmental feedback. Code as Policies [11] generates robot programs that compose perception and control APIs. Inner Monologue [12] uses execution feedback for closed-loop planning. ROS-LLM [13] integrates language models with ROS for multiple execution representations, skill extension, and feedback-driven reflection. RoboClaw [14] uses a VLM to orchestrate learned policy primitives. Recent AgentOS frameworks further combine spatial memory, heterogeneous capability scheduling, and runtime management: HoloAgent-0 [7] supports 3D spatial memory and cross-robot execution, ABot-Claw [8] provides shared cross-embodiment memory and feedback-driven correction, and ABot-AgentOS [15] stores environmental and task context in a source-grounded multimodal graph. These systems support cross-robot coordination and feedback-driven recovery. Table I summarizes the capabilities of representative agentic robotic systems, including collaboration, spatial memory, and the two forms of RSI considered in this work. Despite these advances, mechanisms for turning collaboration failures into reusable handoff checks and recovery rules remain underspecified. InstAgent combines shared observations with collaboration BTs to revise inter-robot dependencies and handoff conditions using execution feedback and retain effective revisions for subsequent tasks.

B. Heterogeneous Multi-Robot Collaboration

LLM-based multi-robot systems organize joint actions through dialogue negotiation or hierarchical task allocation. RoCo [16] negotiates task plans through inter-robot dialogue and refines execution using motion planning and environmental feedback. Chen et al. [17] further compare central-

ized, decentralized, and hybrid communication architectures in terms of task success and communication efficiency. These approaches support flexible and interpretable coordination, although communication and context requirements increase with team size and task horizon. Hierarchical methods separate global coordination from local execution [18]. COHERENT [9], for example, uses a centralized task assigner to match heterogeneous capabilities and adjusts plans through feedback from robot executors.

Executing the allocated tasks also requires organizing action dependencies and recovery procedures. In robotics, behavior trees (BTs) provide a control structure for this purpose [19], [20], combining reactivity and modularity with ease of modification and interpretability [21], [22]. LLM-HBT [10] integrates this structure with task allocation, using condition checks, local subtree extension, and inter-robot task reassignment to recover from failures. These methods improve coordination and runtime adaptation, but leave underspecified how corrected handoff conditions and recovery dependencies are validated, persistently retained, and inherited for further revision across tasks. InstAgent retains validated handoff checks and recovery logic as reusable collaboration subtrees, enabling subsequent tasks to inherit and further improve effective revisions made during execution.

C. RSI and Executable Experience

Existing embodied-agent research has explored how execution feedback can be converted into reusable experience. Reflexion [1] stores feedback as verbal reflections and represents Experience-Context RSI, whereas Voyager [2] and LRLL [3] convert experience into retrievable, composable, and executable skills, representing Structured-Program RSI. However, these methods primarily target a single agent or single-robot skills and do not represent role bindings, handoff conditions, or recovery dependencies in heterogeneous teams.

In the context of physical AgentOS systems, PhyAgentOS [4] organizes execution, verification, diagnosis, revision, re-verification, and experience consolidation into a self-evolution loop. ABot-AgentOS [15] uses failure-driven self-evolution to improve memory writing, retrieval, and evidence-selection policies. These systems primarily represent Experience-Context RSI, rather than directly revising heterogeneous multi-robot collaboration programs.

Experience-Context RSI is flexible, but translating experience into concrete execution still depends on the Agent's current reasoning and does not inherently preserve stable execution orders and explicit dependencies. Structured-Program RSI retains conditions, execution orders, and recovery branches in the program structure, making it better suited to repeated or structurally similar collaboration tasks. InstAgent combines both forms of RSI: Experience-Context RSI uses experience documents, failure feedback, and shared observations to guide revisions, while Structured-Program RSI stores validated revisions as collaboration BT subtrees for subsequent inheritance, rebinding, and improvement.

III. InstAgent: A Self-Improving Agent OS for Heterogeneous Collaboration

System Architecture Overview. InstAgent is a ROS 2-based agentic OS that coordinates heterogeneous robots through BTs. At the lowest level, capabilities are exposed as atomic skills, each wrapped as a minimal Skill BT (see Sec. III-A). Guided by task goals, observations, and retrieved experience, the Agent uses HeteroComposer (see Sec. III-B) to combine these Skill BTs into a collaboration BT that specifies cross-robot dependencies, handoff conditions, and recovery branches. The Coordinator binds its action nodes to available skills, while the Executive executes the tree and returns feedback.

Collab-RSI (see Sec. III-C) is the program-level recursive self-improvement mechanism for collaboration BTs. The Agent uses execution feedback to revise the tree, which resumes execution after passing the required checks. After successful execution and goal verification, the BT is saved as a reusable subtree in the experience library, with a paired document recording its applicability and invocation conditions. Later tasks inherit and adapt these subtrees, writing further validated revisions back for reuse. SMAR-T (see Sec. III-D) supports planning and failure diagnosis by retrieving shared robot observations according to semantic content, location, and time.

A. Skill BTs: From Atomic Skills to Executable Trees

Atomic skills in InstAgent are provided by shared modules and robot modules. Shared modules provide navigation and perception capabilities, while robot modules expose motion, speech interaction, and manipulation capabilities specific to each robot. For executable atomic skills, we represent a concrete skill invocation as a minimal behavior tree containing a single `Action` node, termed a *Skill BT*, which serves as a basic unit for subsequent task composition. The single-`Action` tree is the minimal atomic instance. The Skill BTs category in Fig. 2(a) also includes reusable capability procedures composed from these atomic instances.

To support skill understanding and selection, the multimodal large language model (MLLM) reads functional descriptions, interface definitions, and documented invocation contexts, calling procedures, and usage conditions, and converts them into the common skill representation used by the RSI library. Each experience records the skill's purpose, parameter meanings, and usage constraints, maintaining consistency with the registered interface. The system constructs the minimal Skill BT through a deterministic wrapper: the `skill` field identifies the interface as `module/skill`, and `args` carries the invocation arguments. The resulting tree is stored and linked to its corresponding experience. Before execution, the system checks the BT format, interface existence, and argument consistency with the interface schema. Arguments requiring runtime resolution are checked against the schema once resolved.

During task planning, the Agent selects Skill BTs based on task requirements and skill experiences, and supplies the arguments required by the current task. The selected Skill

BTs serve as basic subtrees for constructing the complete task BT; subtree composition and dependencies across robots are detailed in the next subsection.

B. HeteroComposer: Collaborative BT Construction

HeteroComposer selects reusable programs for the current task, with paired documents and their applicability context. Given a task goal g_k and context $x_{k,0}$, the system retrieves candidate entries from the experience document store D_k. The MLLM examines the goals and usage scope described in these documents to select applicable experience, then retrieves the associated programs from the subtree store $\mathscr{T}_k$. The documents and programs jointly form the experience set:

$$\begin{aligned} \mathscr{D}_k^{\text{cand}} &= \text{RetrieveDocs}(D_k; g_k, x_{k,0}), \\ \mathscr{D}_k^{\text{sel}} &= \text{Select}_{\text{MLLM}}(\mathscr{D}_k^{\text{cand}}; g_k, x_{k,0}), \\ \mathscr{E}_k &= \{(d, \mathscr{T}_k[\ell(d)]) \mid d \in \mathscr{D}_k^{\text{sel}}\}, \end{aligned} \tag{1}$$

where $\ell(d)$ identifies the subtree associated with document d.

The Agent constructs a complete collaborative BT from this experience, prioritizing the reuse of existing program structures and filling uncovered steps with atomic skills. Using the available capability information $\mathscr{S}_k$, the Agent selects robots or modules for individual actions and specifies task-dependent call arguments. Cross-robot coordination is expressed through sequences, conditions, and recovery branches in the BT, allowing subsequent actions to start once their prerequisite requirements are satisfied.

C. Collab-RSI: Experience-Guided BT Evolution for Heterogeneous Collaboration

Collab-RSI enables program-level recursive self-improvement in InstAgent by revising inherited collaboration procedures and their composition using execution feedback, and retaining validated changes for subsequent tasks. When a collaborative BT constructed by HeteroComposer encounters a failed handoff, an unmet execution dependency, or an insufficient recovery procedure, the Agent identifies the affected coordination steps using the current program, task goal, observations, and related experience. Revisions may modify the internal structure of a reused subtree, such as by adding an outcome check, or modify the relationships among subtrees by adjusting action assignments, handoff conditions, and recovery paths across robots. These revisions address whether the outcome produced by one robot satisfies the conditions required by a subsequent robot.

After structural and skill-interface checks, the candidate program is re-executed to verify the relevant handoff conditions and task outcome. When cross-task experience updates are enabled, validated programs are written to the experience library as linked BT structures and experience documents. The BT structures retain inter-robot execution dependencies, handoff checks, and recovery procedures, while the documents record task goals, required capabilities and robot roles, applicability conditions, failure context, and revision rationale. Revisions to existing experience update both the

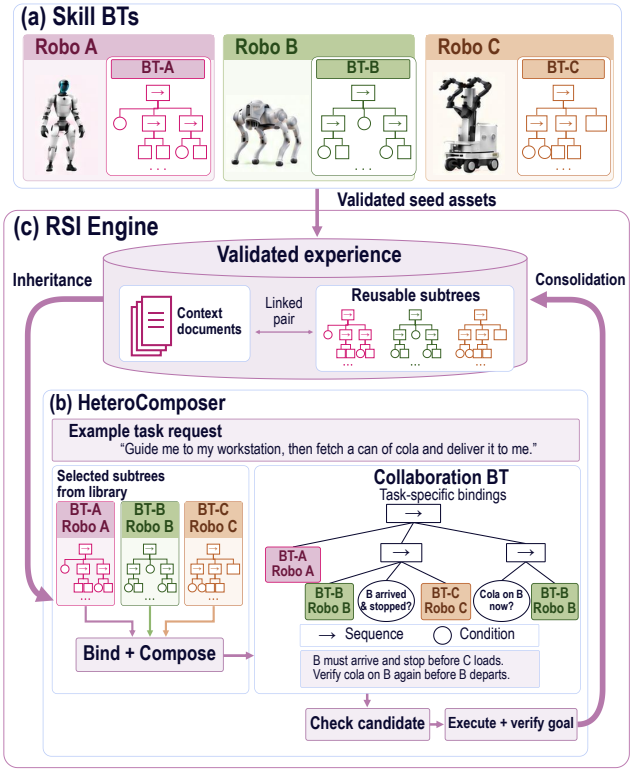


Fig. 2. BT-based RSI from Skill BTs to collaboration programs. (a) Skill BTs provide reusable capability procedures for task composition. (b) HeteroComposer binds retrieved subtrees to task-specific roles and arguments and composes them with inter-robot handoff conditions. (c) The RSI Engine retains validated programs with linked context documents for inheritance and further revision. BT-A, BT-B, and BT-C denote example Skill BTs associated with robots A, B, and C. The service case in Section IV-B reports a human-triggered revision followed by automatic reuse.

associated structure and its document. A validated program composed from multiple existing subtrees is stored as a new experience entry together with its complete coordination structure and applicable context, making the composition itself retrievable and reusable. Failed execution records are retained as diagnostic context for subsequent analysis and applicability assessment. Let D_k and $\mathscr{T}_k$ denote the document collection and the collection of stored BTs and subtrees in the experience library, respectively. With execution record τ_k and validation results v_k, the persistent update is expressed as

$$(D_{k+1}, \mathscr{T}_{k+1}) = \text{Update}(D_k, \mathscr{T}_k, \tau_k, v_k). \tag{2}$$

For a subsequent task, the Agent selects applicable documents from the updated Experience library according to the current goal, the recorded applicability conditions, and the available robot capabilities, and retrieves the linked program structures and context. Through the composition process in HeteroComposer, the Agent can inherit previously revised subtrees or use a validated composite program as a reusable unit, preserving its inter-robot dependencies, handoff checks, and recovery logic while adapting it to the current task. New feedback may reveal changed coordination conditions or additional handoff failures, prompting further revisions

Algorithm 1 Collab-RSI: Experience-Guided BT Evolution

Require: Goal g_k, capabilities $\mathscr{S}_k$, stores $D_k, \mathscr{T}_k$
Require: Recovery budget B, candidate limit $J_{\max} = 3$, write flag w
Ensure: Outcome o, trace τ_k, stores $D_{k+1}, \mathscr{T}_{k+1}$

Observe x; $\tau_k \leftarrow \emptyset$; $o \leftarrow$ INCOMPLETE
$\mathscr{E}_k \leftarrow \text{SelectExperience}(D_k, \mathscr{T}_k; g_k, x)$
$T \leftarrow \text{ComposeAndCheck}(g_k, x, \mathscr{E}_k, \mathscr{S}_k)$
if T is unavailable **then return** $o, \tau_k, D_k, \mathscr{T}_k$
while true **do**
    $(o, e, x, \tau_k, v_k, B) \leftarrow$
        $\text{ExecuteAndVerify}(T, g_k, \tau_k, B)$
    if $o =$ SUCCESS or $B = 0$ or e is terminal **then break**
    if safe stopping is unconfirmed **then break**
    $T' \leftarrow \text{ReviseAndCheck}_{J_{\max}}(T, \mathscr{E}_k, g_k, x, e)$
    if T' is unavailable **then break**
    $T \leftarrow \text{RebuildWithCompletionChecks}(T')$; $B \leftarrow B - 1$
end while
$(D_{k+1}, \mathscr{T}_{k+1}) \leftarrow (D_k, \mathscr{T}_k)$
if $w = 1$ **then**
    $(D_{k+1}, \mathscr{T}_{k+1}) \leftarrow \text{Update}(D_k, \mathscr{T}_k, \tau_k, v_k)$
end if
return $o, \tau_k, D_{k+1}, \mathscr{T}_{k+1}$

whose validated results are written back to the same library. A composite program formed in one task thus becomes inherited experience in later tasks and remains available for further reuse, composition, and revision. This closes the recursive loop of inheritance, execution, revision, validation, storage, and subsequent inheritance, accumulating executable experience for collaboration among robots with different capabilities.

D. SMAR-T: Shared Multimodal Spatio-Temporal Memory with Agent-Driven Retrieval

SMAR-T pools robot observations in environmental memory $\mathscr{H}$ to support collaborative planning and failure diagnosis. These observations complement experience documents D_k and reusable subtrees $\mathscr{T}_k$: they inform BT construction and revision, while Collab-RSI retains validated programs and their context. Observations from one robot can thus support subsequent tasks involving another.

Shared observation memory. Each entry records an image I_i, its caption c_i, visual and caption embeddings z_i^v, z_i^c, the observing robot's pose $p_i = (x_i, R_i)$ in the shared map, comprising position $x_i \in \mathbb{R}^3$ and orientation $R_i \in \mathrm{SO}(3)$, timestamp t_i, and source robot r_i:

$$\begin{aligned} c_i &= G_{\text{cap}}(I_i), \\ z_i^v &= \text{norm}(E_v(I_i)), \\ z_i^c &= \text{norm}(E_t(c_i)), \\ m_i &= (I_i, c_i, z_i^v, z_i^c, p_i, t_i, r_i). \end{aligned} \tag{3}$$

Here, m_i denotes one observation, and G_{cap} uses a vision-language model (VLM) to generate a caption. The visual–text encoding follows standard multimodal practice [23]–[25]. Shared-map poses, timestamps, and source identities link observations across robots and task stages. For example, comparing loading and delivery observations helps diagnose missing-payload failures and informs revisions to handoff checks and recovery branches.

Agent-driven retrieval and evaluation loop. Figure 3 illustrates the loop. The Agent identifies semantic, spatial, and temporal constraints, then selects a read-only tool and search domain $\mathscr{U} \subseteq \mathscr{H}$. Each call may search the full memory or refine a returned candidate pool, including returning to the full memory on a later call. Tool order depends on the task; the illustrated request establishes the living-room and recent-time scope before retrieving the red-sofa scene.

After each call, the Agent jointly evaluates images, captions, and metadata against the original constraints and updates its task context. Ranking alone does not establish constraint satisfaction. The loop succeeds when evidence supports all explicit constraints and supplies the required response fields; conflicts, missing fields, or unresolved references prompt further retrieval. Each real-robot query allows at most 10 calls across the three tools. If evidence remains insufficient and the pool is empty or the budget is exhausted, the Agent reports insufficient evidence and may request a fresh observation. Sufficient evidence supports a grounded response and, when requested, BT construction or revision. The Coordinator binds actions to Skills, and the Executive checks arguments and executes the BT, using current observations to verify physical conditions.

Semantic, spatial, and temporal retrieval tools. All tools search $\mathscr{U}$ and preserve each candidate's image, caption, pose, timestamp, and source. The position tool ranks by $d_p(i \mid \hat{x}) = \|x_i - \hat{x}\|_2$ for query location $\hat{x}$, using only the position component of p_i; orientation is retained for navigation. The time tool ranks by $d_t(i \mid \hat{t}) = |\Delta t_i - \hat{t}|$, with observation and query times measured from the same scene origin.

For semantic query q, the semantic tool first recalls at most ten frames by visual similarity, then reranks only these frames using their captions:

$$\begin{aligned} z_q &= \text{norm}(E_t(q)), \\ v_i &= \cos(z_q, z_i^v), \quad u_i = \cos(z_q, z_i^c), \\ \mathscr{C}_{10} &= \text{TopK}_{10, m_i \in \mathscr{U}}(v_i), \\ s_i &= (1-\lambda)\widetilde{v}_i + \lambda \widetilde{u}_i, \quad \mathscr{C}_5 = \text{TopK}_{5, m_i \in \mathscr{C}_{10}}(s_i). \end{aligned} \tag{4}$$

The tildes denote min–max normalization within $\mathscr{C}_{10}$, and $\lambda = 0.15$. The tool returns at most five candidates for Agent evaluation.

IV. EXPERIMENTS

We evaluate BT-based execution, revision, and procedural-experience reuse through real-robot comparisons and a service case, and assess task-relevant environmental context through RAVEN-QA retrieval.

A. Experimental Setup

Platforms, tasks, and paired trials. G1, Go2, and ALOHA provide interaction and guidance, navigation and transport, and manipulation and loading, respectively; shared navigation and perception modules return state and execution feedback. We evaluate 20 heterogeneous multi-robot tasks involving capability allocation and inter-robot handoffs. Recurring collaboration structures vary in objects, locations,

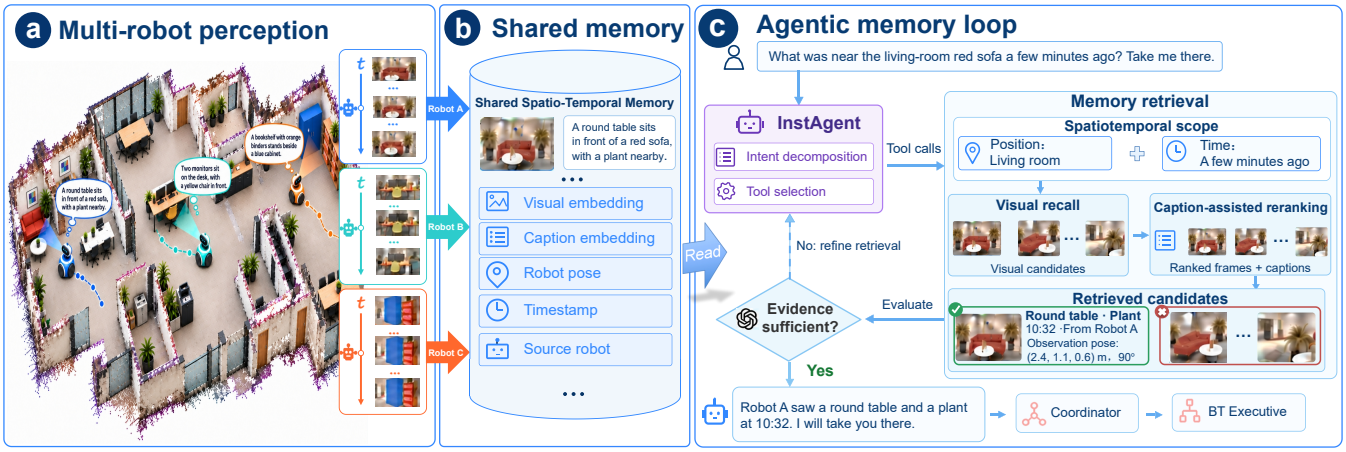


Fig. 3. SMAR-T shared memory and the InstAgent memory loop. (a) Multiple robots collect observations in a shared map. (b) Each memory entry couples an image and its caption with visual and caption embeddings, the observing robot's pose, timestamp, and source identity. (c) InstAgent decomposes the request and selects retrieval tools. In the illustrated example, spatial and temporal constraints define the retrieval scope, followed by visual recall and caption-assisted reranking. The Agent evaluates the returned candidates and their metadata against the request. Insufficient evidence triggers further retrieval; sufficient evidence supports a grounded response and, when requested, task execution through the Coordinator and BT Executive.

and goals, testing subtree reuse and adaptation within a fixed robot team under the perturbations below. Each of three models runs every task twice under four configurations: 40 runs per model–configuration and 480 measured runs in total. For each model, corresponding task repetitions are paired across configurations, with task instances, failure settings, and perturbation types, injection counts, and trigger stages held constant.

Success criteria and recovery budgets. Perturbations include excessive arrival error, navigation and loading failures, and user-requested destination changes. Relative movement and turning must be within 0.10 m and 10°; workstation navigation requires a successful module return and human confirmation of arrival within 0.5 m. Success requires all physical and interaction postconditions to be verified. Each task permits at most five executed failure-triggered recoveries or retries; a goal change alone consumes no retry. For BT conditions, each failure episode additionally permits three candidate revisions, with the counter reset per episode. Rejected candidates consume candidate attempts, not executed-recovery budget. An incomplete run fails when the recovery budget is exhausted or, if BT revision is required, all three candidates are rejected or safe stopping cannot be confirmed (Algorithm 1).

Configurations and experience control. Simple CoT invokes Skills from a textual plan without a BT, under the same tasks, success criteria, and recovery budget. **BT Recovery** (E_0) builds BTs from registered atomic Skills without cross-task subtree retrieval. **Fixed-Experience BT** (E_1) additionally retrieves existing subtrees while preserving their executable versions. **RSI-Updated Experience BT (E_2, full InstAgent)** starts from the same base subtrees as E_1, but uses matched historical feedback and case availability to revise, validate, and retain them before evaluation.

All BT conditions allow the same failure recovery, task-local structural revision, and retries under identical budgets. Each model has its own procedural library; the E_1 and E_2 snapshots are prepared before evaluation and frozen during measured runs. Task-local repairs are not written back, so one run cannot update another's library. Evaluation tasks are excluded from library construction. Tasks, models, prompts, perturbations, and recovery budgets are shared across BT conditions; E_1 and E_2 also use the same retrieval procedure. Thus, E_0–E_1 tests experience reuse, whereas E_1–E_2 tests retention of validated revisions. Model parameters, atomic Skills, low-level controllers, VLA/VLM components, and task specifications remain fixed.

Metrics. Task success rate (SR) measures complete task success. First-attempt success rate (FSR) requires success without failure-triggered recovery or retry; a user-requested goal change alone does not count as a retry. Initial-BT logical coherence (LC) follows the precondition-respecting execution-order criterion of [26] for action, stage, and hand-off dependencies. Alternative valid orders are accepted, repaired trees are excluded, and LC does not apply to Simple CoT. Mean recovery count ($\bar{R}$) averages failure-triggered recoveries and retries over all runs, including ultimately unsuccessful ones.

RAVEN-QA evaluation protocol. RAVEN-QA [6] provides sparse observations with timestamps and poses from RealWorld (robot exploration in laboratories and public areas), Habitat-Sim (simulated robot-view trajectories), IRS (indoor/outdoor web videos), and IRS-hard (challenging self-recorded/web egocentric videos). Queries cover object retrieval, visual and commonsense reasoning, information recall, and spatial understanding. We run RAVEN [6], ReMEmbR [5], and both InstAgent variants under the official RAVEN-QA protocol, using identical splits, queries, preprocessing, QQMM-v2 embeddings [25], success criteria, and evaluation scripts. RAVEN and ReMEmbR retain their

TABLE II

REAL-ROBOT PERFORMANCE ACROSS HIGH-LEVEL MODELS, EXECUTION CONFIGURATIONS, AND PROCEDURAL-EXPERIENCE STAGES.

Configuration	SR↑	FSR↑	LC↑	$\bar{R}\downarrow$
GPT-5.6 Sol				
Simple CoT	25/40	20/40	—	1.60
BT Recovery (E_0)	32/40	30/40	35/40	1.10
Fixed-Experience BT (E_1)	34/40	32/40	37/40	0.85
Full InstAgent (E_2)	38/40	35/40	39/40	0.40
DeepSeek-V4-Pro				
Simple CoT	22/40	14/40	—	2.15
BT Recovery (E_0)	29/40	24/40	33/40	1.65
Fixed-Experience BT (E_1)	31/40	28/40	35/40	1.30
Full InstAgent (E_2)	38/40	36/40	39/40	0.35
Qwen3.8-27B				
Simple CoT	17/40	11/40	—	2.70
BT Recovery (E_0)	24/40	21/40	28/40	2.20
Fixed-Experience BT (E_1)	28/40	24/40	32/40	1.80
Full InstAgent (E_2)	34/40	29/40	36/40	1.00

Each model–configuration pair includes 40 real-robot runs (20 tasks, two runs per task). SR, FSR, and LC are counts out of 40; $\bar{R}$ averages failure-triggered recoveries and retries over all runs. All BT conditions allow task-local revision under the same recovery budget. E_1 and E_2 share historical feedback and cases; only E_2 persists validated revisions to inherited subtrees before evaluation. Both procedural libraries remain frozen during evaluation. LC evaluates initial BTs and does not apply to Simple CoT (—).

TABLE III

QUERY-LEVEL TASK SUCCESS RATE (SR, %) ON RAVEN-QA. HIGHER IS BETTER.

Subset	RAVEN	ReMEmbR	Ours	Ours w/o Agent
RealWorld	89.05	76.67	**95.24**	76.19
Habitat-Sim	80.47	54.99	**93.79**	63.35
IRS	96.30	92.60	**98.15**	92.59
IRS-hard	82.90	58.50	**82.93**	58.54
Overall	84.57	64.49	**93.14**	69.31

All results are from our own runs. RAVEN, ReMEmbR, and Ours use QQMM-v2 and Gemini-2.5-Flash. Ours w/o Agent retains the same embedder and directly returns the Top-1 retrieval result without a VLM Agent.

respective retrieval protocols; model configurations appear in Table III. Query-level SR is the percentage of queries with the requested observation or associated answer correctly identified. Full InstAgent uses Gemini-2.5-Flash for tool selection, evidence interpretation, and answer generation. *Ours w/o Agent* retains the same indexed observations, captions, embeddings, visual recall, and caption reranking, but returns Top-1 directly. The full system adds candidate comparison, position or time retrieval when needed, and caption verification against source images.

B. Model Comparison and BT-Based RSI Evaluation

Execution and experience. Table II compares three high-level models using Simple CoT and three BT experience stages. Baseline planning and atomic Skills can complete some tasks. Across all three models, Full InstAgent achieves higher SR and FSR and fewer recoveries than BT Recovery, supporting the benefit of retained and updated procedural experience on the 20 evaluated tasks.

Experience-stage progression. Table II shows performance across fixed experience stages. The primary contrast is E_0–E_2: across the three models, feedback-updated experience improves SR by 15, 22.5, and 25 points, FSR by 12.5, 30, and 20 points, and reduces $\bar{R}$ by 0.70, 1.30, and 1.20. We compare E_0 and E_2 at the task-instance level using McNemar's test for paired task-success outcomes and the Wilcoxon signed-rank test for paired recovery counts. Both tests yield $p < 0.05$ for all three models. The E_0–E_1 comparison evaluates the benefit of reusing existing procedural experience. The E_0–E_2 contrast measures the combined benefit of experience reuse and retained revisions. With base subtrees, historical feedback, and case availability matched, E_1–E_2 assesses the additional benefit associated with retaining validated revisions in the executable subtrees. These comparisons assess program reuse and adaptation across object, location, and goal variations using prepared library snapshots. Relative to E_1, the observed SR gains of E_2 are 10, 17.5, and 15 percentage points for GPT-5.6 Sol, DeepSeek-V4-Pro, and Qwen3.8-27B, respectively. LC also increases monotonically from E_0 to E_2 across all three models.

Collaboration-program revision and reuse. This qualitative case, excluded from the aggregate results, illustrates handoff diagnosis, program revision, and subsequent reuse. After G1 guides the user to a workstation, Go2 collects cola from ALOHA and returns after loading verification. Shared item, loading-location, and destination arguments connect the capability subtrees.

In one run, cola was manually removed after loading confirmation but before Go2 departed. After arrival, a tester's web-interface report of missing cola prompted the Agent to retrieve historical observations and identify its absence during transport. Reflection and replanning added VLM checks of current basket contents by ALOHA before return navigation and G1 before delivery; either failed check triggers reloading. The validated subtree retains both checks and recovery logic for automatic reuse, while its linked document records the diagnosis and revision rationale (Fig. 2). Atomic Skills remain unchanged.

C. RAVEN-QA Memory Evaluation

The methods differ in how they represent and select visual experience. RAVEN directly indexes original observations with multimodal visual embeddings, preserving image details but using no language description as a second ranking signal [6]. ReMEmbR retrieves caption-based memories [5], making semantics explicit but losing details omitted by captioning. Our method combines their complementary signals: visual similarity first forms a high-recall candidate set, captions refine its order, and the Agent verifies the selected captions against their original images. Language thus assists selection without replacing visual evidence.

Our method reaches 93.14% overall, exceeding RAVEN by 8.57 points and ReMEmbR by 28.65 points. Its largest

gains over RAVEN occur on RealWorld (6.19 points) and Habitat-Sim (13.32 points), consistent with caption reranking helping distinguish visually similar scenes by object identity, attributes, or role. IRS leaves only a 1.85-point margin because direct visual retrieval is already strong. The near tie on IRS-hard (82.93% versus 82.90%) exposes the method's boundary: reranking cannot recover targets that are absent, occluded, or missed by the visual candidate set.

Returning Top-1 without the Agent reduces overall SR from 93.14% to 69.31%, with the largest drops on Habitat-Sim and IRS-hard. The Agent is a unified retrieval controller that performs candidate comparison, position and time tool calls, evidence interpretation, and caption verification. Thus, this 23.83-percentage-point gap measures the overall contribution of the integrated Agent-driven retrieval workflow rather than the contribution of any individual component. The smaller IRS drop is consistent with its more direct queries, for which Top-1 is more often sufficient.

V. Conclusion

InstAgent connects capability binding, shared context, and physical execution through an agent operating system with BT-based RSI. Linked documents and executable subtrees retain feedback-driven changes to collaboration programs, allowing inherited procedures to be composed and revised again. Real-robot evaluation across three models shows higher task and first-attempt success with feedback-updated experience than with cold starts on the evaluated tasks. The service case illustrates how a retained BT condition changes the handoff between loading and transport. Future work will examine transfer to new task compositions and robot teams, and the reliability of experience updates over longer deployment periods.

References

[1] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," *Advances in neural information processing systems*, vol. 36, pp. 8634–8652, 2023.

[2] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *arXiv preprint arXiv:2305.16291*, 2023.

[3] G. Tziafas and H. Kasaei, "Lifelong robot library learning: Bootstrapping composable and generalizable skills for embodied control with language models," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 515–522.

[4] Y. Liu, W. Chen, X. Song, T. Pu, S. Mo, Y. Bai, Z. Chen, Q. Sun, L. Zhong, Y. Shen, *et al.*, "Phyagentos: A self-evolving operating system for embodied agents with decoupled cognitive planning and physical execution," *arXiv preprint arXiv:2607.16636*, 2026.

[5] A. Anwar, J. Welsh, J. Biswas, S. Pouya, and Y. Chang, "Remembr: Building and reasoning over long-horizon spatio-temporal memory for robot navigation," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 2838–2845.

[6] Y. Hu, Z. Zheng, L. Zha, C. Xing, R. Singh, O. Hossain, A. Loquercio, and D. Shah, "Raven: Long-horizon reasoning & navigation with a visuo-spatio-temporal memory," *arXiv preprint arXiv:2606.25206*, 2026.

[7] X. Zhou, L. Liu, T. Xiao, W. Feng, F. Fu, X. Meng, X. Wang, J. Han, B. Yu, Y. Du, *et al.*, "Holoagent-0: A unified embodied agent framework with 3d spatial memory," *arXiv preprint arXiv:2606.23565*, 2026.

[8] D. Huo, H. Liu, G. Liu, D. Qi, Z. Sun, M. Gao, J. He, Y. Yang, X. Chang, F. Xiong, *et al.*, "Abot-claw: A foundation for persistent, cooperative, and self-evolving robotic agents," *arXiv preprint arXiv:2604.10096*, 2026.

[9] K. Liu, Z. Tang, D. Wang, Z. Wang, X. Li, and B. Zhao, "Coherent: Collaboration of heterogeneous multi-robot system with large language models," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 10 208–10 214.

[10] C. Wang, J. Sun, Y. Zhang, M. Zhang, and C. Wu, "Llm-hbt: Dynamic behavior tree construction for adaptive coordination in heterogeneous robots," *arXiv preprint arXiv:2510.09963*, 2025.

[11] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, "Code as policies: Language model programs for embodied control," in *2023 IEEE International conference on robotics and automation (ICRA)*. IEEE, 2023, pp. 9493–9500.

[12] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, *et al.*, "Inner monologue: Embodied reasoning through planning with language models," *arXiv preprint arXiv:2207.05608*, 2022.

[13] C. E. Mower, Y. Wan, H. Yu, A. Grosnit, J. Gonzalez-Billandon, M. Zimmer, P. Liu, D. Palenicek, D. Tateo, J. Peters, *et al.*, "A robot operating system framework for using large language models in embodied ai," *Nature Machine Intelligence*, vol. 8, no. 3, pp. 313–325, 2026.

[14] R. Li, Y. Zhou, Y. Zhu, K. Chen, J. Wang, S. Wang, K. Hu, M. Yu, B. Jiang, Z. Su, *et al.*, "Roboclaw: An agentic framework for scalable long-horizon robotic tasks," *arXiv preprint arXiv:2603.11558*, 2026.

[15] J. Tian, S. Liu, Y. Xu, J. Lu, Z. Guan, H. Han, D. Yang, M. Gu, Y. Qian, T. Zhang, *et al.*, "Abot-agentos: A general robotic agent os with lifelong multi-modal memory," *arXiv preprint arXiv:2607.10350*, 2026.

[16] Z. Mandi, S. Jain, and S. Song, "Roco: Dialectic multi-robot collaboration with large language models," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 286–299.

[17] Y. Chen, J. Arkin, Y. Zhang, N. Roy, and C. Fan, "Scalable multi-robot collaboration with large language models: Centralized or decentralized systems?" in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 4311–4317.

[18] X. Zhang, H. Qin, F. Wang, Y. Dong, and J. Li, "Lamma-p: Generalizable multi-agent long-horizon task allocation and planning with lm-driven pddl planner," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 10 221–10 221.

[19] M. Colledanchise, D. Almeida, and P. Ögren, "Towards blended reactive planning and acting using behavior trees," in *2019 international conference on robotics and automation (ICRA)*. IEEE, 2019, pp. 8839–8845.

[20] H. Zeng, M. Wang, and P. Li, "Emboteam: Grounding llm reasoning into reactive behavior trees via pddl for embodied multi-robot collaboration," *arXiv preprint arXiv:2601.11063*, 2026.

[21] M. Iovino, E. Scukins, J. Styrud, P. Ögren, and C. Smith, "A survey of behavior trees in robotics and ai," *Robotics and Autonomous Systems*, vol. 154, p. 104096, 2022.

[22] Y. Cai, X. Chen, Y. Mao, M. Li, S. Yang, W. Yang, and J. Wang, "Hbtp: Heuristic behavior tree planning with large language model reasoning," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 13 706–13 713.

[23] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, *et al.*, "Learning transferable visual models from natural language supervision," in *International conference on machine learning*. PmLR, 2021, pp. 8748–8763.

[24] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, "Sigmoid loss for language image pre-training," in *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, 2023, pp. 11 941–11 952.

[25] Y. Xue, D. Li, and G. Liu, "Improve multi-modal embedding learning via explicit hard negative gradient amplifying," *arXiv preprint arXiv:2506.02020*, 2025.

[26] J. Ao, F. Wu, Y. Wu, A. Swiki, and S. Haddadin, "Llm-as-bt-planner: Leveraging llms for behavior tree generation in robot task planning," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 1233–1239.